DRIVER SAFETY PROJECT

IR-Based Driver Drowsiness Detection & Alarm System

Real-time eye-closure monitoring that sounds an alarm the moment a driver shows signs of fatigue — before drowsiness turns into an accident.



THE PROBLEM

Driver Fatigue Is a Silent Road Hazard

40%+

Of road accidents worldwide are linked to driver fatigue and micro-sleep

3–5 sec

Of continuous eye closure is the danger zone that must trigger an alert

0

Alerts most vehicles give the driver before a fatigue-related crash

Existing vehicles rarely monitor the driver at all — they watch the road, not the person behind the wheel. By the time a car drifts out of its lane, it is often already too late to react.

RELATED WORK

Proven Technology, Locked Behind a High Price

IR-based driver drowsiness detection already exists — but almost entirely inside premium OEM vehicles and commercial fleet contracts. This project brings the same proven idea to the individual owner.

CURRENT MARKET LANDSCAPE

Premium & Mid-High OEM Cars Only

Toyota, Volvo, Mercedes-Benz and others build IR driver-monitoring into higher-end models, not entry-level vehicles

Aftermarket Exists — But Targets Fleets

Bosch, Seeing Machines and Smart Eye sell standalone IR units, priced and marketed for commercial trucking fleets

Regulation Covers New Models Only

The EU mandate applies to newly launched vehicles — it does nothing for the millions already on the road

Research Rarely Reaches Consumers

Academic IR-threshold designs are proven in papers but almost never sold as an affordable personal product

THE GAP THIS PROJECT TARGETS

Built for the Individual Owner

Sized and priced for one person's car or two-wheeler — not a fleet-wide commercial contract

Low-Cost to Build & Sell

Uses affordable Arduino / Raspberry Pi hardware instead of expensive OEM sensor suites

Fits Any Vehicle, New or Old

Dashboard-mountable — works on vehicles that will never get OEM drowsiness detection

Sellable as a Standalone Product

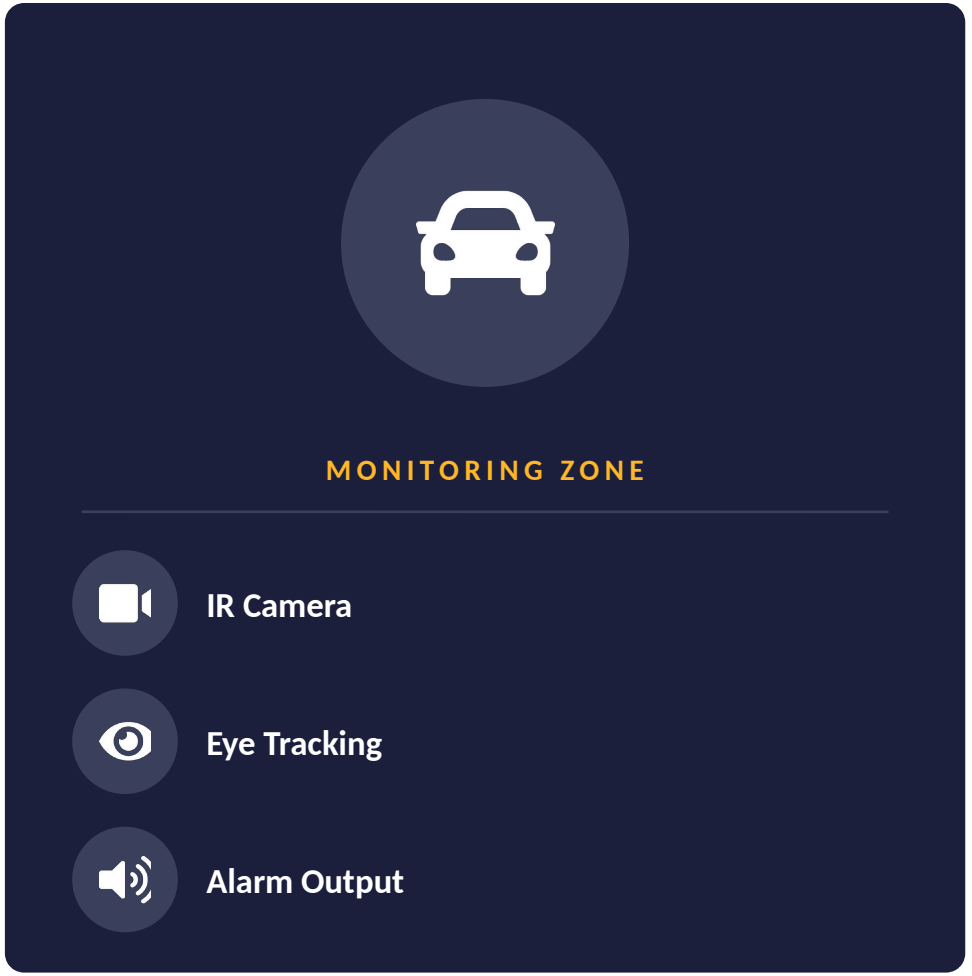
A real, underserved gap: affordable personal-use safety hardware, not locked behind a luxury trim or fleet deal

THE SOLUTION

An IR Camera That Watches the Driver

An infrared (IR) camera is mounted on the dashboard, facing the driver at all times — day or night. It continuously tracks the driver's eyes. The moment the eyes remain continuously closed for more than 3 to 5 seconds, the system treats this as drowsiness and instantly triggers an audible alarm to wake and alert the driver.

- Works in complete darkness using infrared light — invisible to the driver
- Triggers the alarm once eye closure crosses 3–5 continuous seconds
- Sounds an immediate alarm to snap the driver back to full alertness



HOW IT WORKS

System Architecture

1



IR Camera

Continuously captures the driver's face, day or night

2



Eye Detection

Processor locates eyes and checks open / closed state

3



Timer Check

Measures how long eyes remain continuously closed

4



3–5 Sec Threshold

If closure goes beyond 3–5 seconds, drowsiness is flagged

5



Alarm Triggered

Buzzer / alert sounds instantly to wake the driver

The full cycle — capture, detect, time, decide, alert — runs continuously in real time while the vehicle is in motion.

HARDWARE

Components Used



IR Camera Module

Captures driver's face clearly in low light or total darkness



Microcontroller

Raspberry Pi / Arduino runs the eye-detection logic in real time



Buzzer / Alarm Unit

Emits a loud audible alert when drowsiness is detected



Power Supply

Draws power from the vehicle's 12V system via a regulator



Mounting Bracket

Positions the camera on the dashboard facing the driver



Enclosure

Protects electronics from dust, vibration, and heat

METHODOLOGY

Detection Algorithm, Step by Step

1. Capture live video frames continuously from the dashboard-mounted IR camera.
2. Detect the driver's face, then isolate the eye region within each frame.
3. Classify each frame as "eyes open" or "eyes closed" using the eye state.
4. Start an internal timer the moment eyes are detected as continuously closed.
5. If closure lasts more than 3–5 seconds without reopening, flag it as drowsiness.
6. Instantly trigger the buzzer / alarm and reset the timer once eyes reopen.

WHY IT WORKS

Advantages of the System

Day & Night Monitoring

IR sensing means detection quality never drops after dark

Real-Time Response

Alarm fires within seconds of sustained eye closure

Low False Alarms

3–5 sec threshold means normal blinking never triggers it

Accident Prevention

Wakes the driver before fatigue leads to a collision

Low Cost to Build

Uses affordable, widely available components

Fits Any Vehicle

Compact, dashboard-mountable design for cars and trucks

CONCLUSION

A Simple System With Real Impact

This IR camera-based drowsiness detection system gives every vehicle a low-cost, always-on driver monitor. By watching for more than 3–5 seconds of continuous eye closure and sounding an immediate alarm, it turns a silent risk into an early warning — with the potential to prevent countless fatigue-related accidents.

FUTURE SCOPE

- GPS integration to auto-alert nearby contacts on repeated drowsiness
- Cloud logging of driver alertness for fleet safety analytics
- Mobile app notifications paired with in-cabin alarm
- Machine-learning models to predict fatigue before eyes fully close

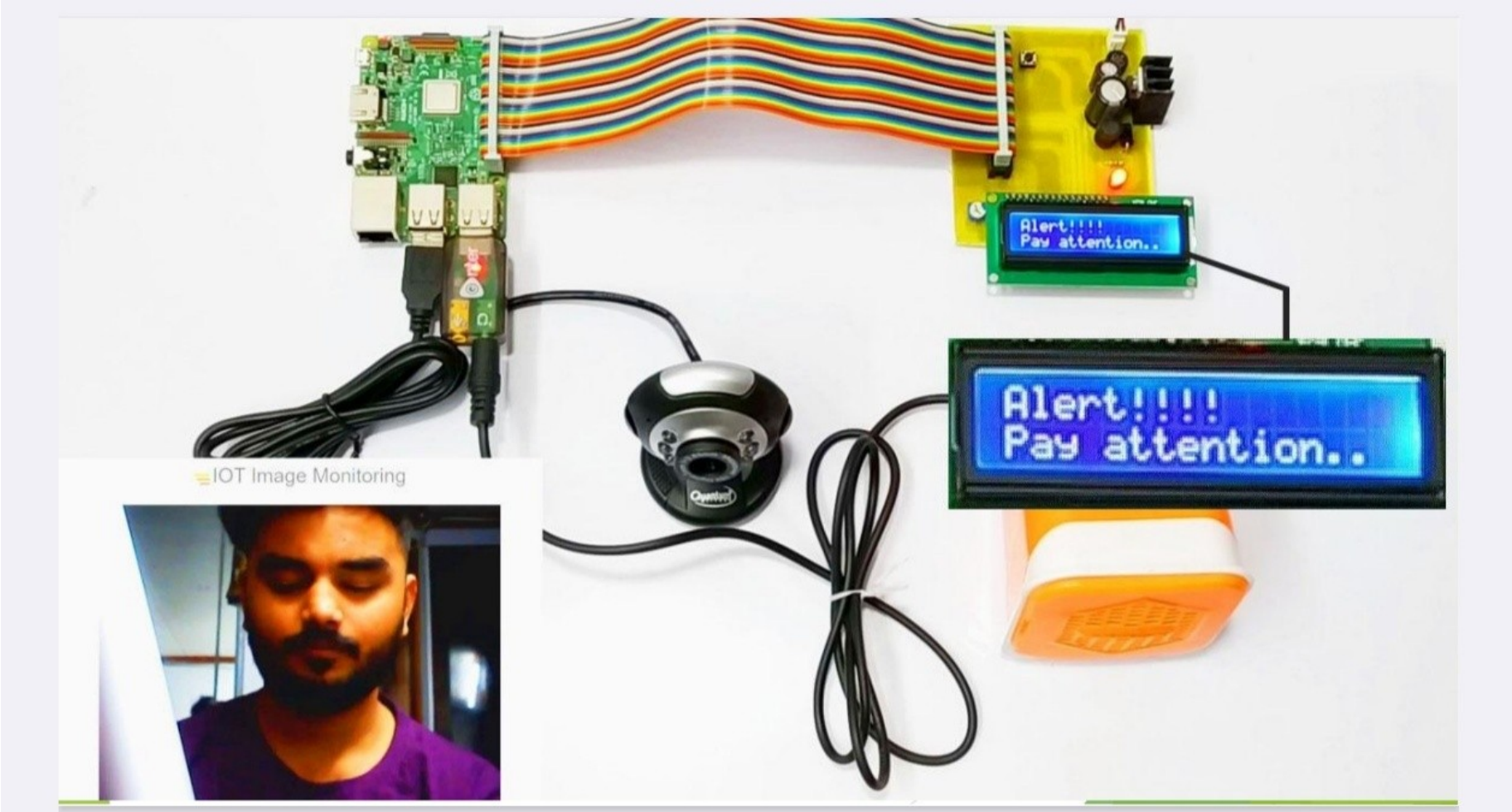


DRIVE SAFE.
ARRIVE SAFE.

Real-time fatigue detection, built for every driver.

HARDWARE

Circuit Diagram



Arduino Code — IR-Based Driver Drowsiness Detection & Alarm System

```
// IR Based Driver Drowsiness Detection and Alarm System

const int IR_SENSOR = 2;
const int BUZZER = 8;
const int RED_LED = 9;
const int YELLOW_LED = 10;
const int GREEN_LED = 11;

unsigned long eyeClosedStart = 0;
unsigned long eyeClosedDuration = 0;

bool eyeClosed = false;

void setup() {

pinMode(IR_SENSOR, INPUT);

pinMode(BUZZER, OUTPUT);
pinMode(RED_LED, OUTPUT);
pinMode(YELLOW_LED, OUTPUT);
pinMode(GREEN_LED, OUTPUT);

digitalWrite(BUZZER, LOW);

Serial.begin(9600);

Serial.println("Driver Drowsiness Detection System");
Serial.println("System Started...");
}

void loop() {

int sensor = digitalRead(IR_SENSOR);

// Eye closed
if (sensor == LOW) {

if (eyeClosed == false) {

eyeClosed = true;
eyeClosedStart = millis();

Serial.println("Eye Closed");
}

eyeClosedDuration = millis() - eyeClosedStart;

// Level 1: Eye closed for 1 second
if (eyeClosedDuration >= 1000 &&
eyeClosedDuration < 2000) {

digitalWrite(GREEN_LED, LOW);
digitalWrite(YELLOW_LED, HIGH);
digitalWrite(RED_LED, LOW);

tone(BUZZER, 1000, 100);

Serial.println("WARNING: Driver may be sleepy");
}

// Level 2: Eye closed for 2 seconds
else if (eyeClosedDuration >= 2000 &&
eyeClosedDuration < 4000) {

digitalWrite(GREEN_LED, LOW);
digitalWrite(YELLOW_LED, LOW);
digitalWrite(RED_LED, HIGH);

tone(BUZZER, 1500);

Serial.println("DROWSINESS DETECTED!");
}

// Level 3: Eye closed for 4 seconds
else if (eyeClosedDuration >= 4000) {
```

```
digitalWrite(GREEN_LED, LOW);
digitalWrite(YELLOW_LED, LOW);
digitalWrite(RED_LED, HIGH);

tone(BUZZER, 2500);

Serial.println("DANGER! DRIVER NOT RESPONDING!");
}
}

// Eye open
else {

if (eyeClosed == true) {

Serial.println("Eye Open");
}

eyeClosed = false;
eyeClosedDuration = 0;

noTone(BUZZER);

digitalWrite(RED_LED, LOW);
digitalWrite(YELLOW_LED, LOW);
digitalWrite(GREEN_LED, HIGH);
}

delay(50);
}
```

Additional Arduino Code — Enhanced Features

This section adds functionality without repeating the original program: startup self-test, warning stages, alarm patterns, closure counting, status monitoring, and reset handling.

```
// Additional Arduino Code
// Enhanced IR Driver Drowsiness Detection System
// Features: startup test, alarm pattern, blink counting,
// cooldown period, system status, and sensor calibration.

const int IR_SENSOR = 2;
const int BUZZER = 8;
const int RED_LED = 9;
const int YELLOW_LED = 10;
const int GREEN_LED = 11;

const unsigned long WARNING_TIME = 1000;
const unsigned long DROWSY_TIME = 2000;
const unsigned long DANGER_TIME = 4000;
const unsigned long RESET_TIME = 500;
const unsigned long ALARM_TIME = 3000;

unsigned long closedStart = 0;
unsigned long alarmStart = 0;
unsigned long lastBlinkTime = 0;

bool eyeClosed = false;
bool alarmActive = false;
int blinkCount = 0;

void allLEDsOff() {
digitalWrite(RED_LED, LOW);
digitalWrite(YELLOW_LED, LOW);
digitalWrite(GREEN_LED, LOW);
}

void startupTest() {
digitalWrite(GREEN_LED, HIGH);
delay(300);
digitalWrite(GREEN_LED, LOW);

digitalWrite(YELLOW_LED, HIGH);
delay(300);
digitalWrite(YELLOW_LED, LOW);

digitalWrite(RED_LED, HIGH);
tone(BUZZER, 1500, 300);
delay(400);
digitalWrite(RED_LED, LOW);

noTone(BUZZER);
}

void setup() {
pinMode(IR_SENSOR, INPUT);
pinMode(BUZZER, OUTPUT);
pinMode(RED_LED, OUTPUT);
pinMode(YELLOW_LED, OUTPUT);
pinMode(GREEN_LED, OUTPUT);

Serial.begin(9600);

allLEDsOff();
startupTest();

digitalWrite(GREEN_LED, HIGH);

Serial.println("================================");
Serial.println("IR DRIVER DROWSINESS SYSTEM");
Serial.println("System Ready");
Serial.println("================================");
}

void warningAlarm() {
tone(BUZZER, 1000, 150);
}

void drowsinessAlarm() {
tone(BUZZER, 1800);
}

void dangerAlarm() {
tone(BUZZER, 2500);
}
```

```
void loop() {

int sensorValue = digitalRead(IR_SENSOR);

// LOW = eye closed for this sensor configuration
if (sensorValue == LOW) {

if (!eyeClosed) {
eyeClosed = true;
closedStart = millis();

// Count a blink/closure event
if (millis() - lastBlinkTime > 300) {
blinkCount++;
lastBlinkTime = millis();
}

Serial.print("Eye closed. Closure count: ");
Serial.println(blinkCount);
}

unsigned long closedTime = millis() - closedStart;

if (closedTime >= WARNING_TIME &&
closedTime < DROWSY_TIME) {

digitalWrite(GREEN_LED, LOW);
digitalWrite(YELLOW_LED, HIGH);
digitalWrite(RED_LED, LOW);

warningAlarm();

Serial.println("WARNING: Possible fatigue");
}

else if (closedTime >= DROWSY_TIME &&
closedTime < DANGER_TIME) {

digitalWrite(GREEN_LED, LOW);
digitalWrite(YELLOW_LED, LOW);
digitalWrite(RED_LED, HIGH);

drowsinessAlarm();

Serial.println("ALERT: DROWSINESS DETECTED");
}

else if (closedTime >= DANGER_TIME) {

digitalWrite(GREEN_LED, LOW);
digitalWrite(YELLOW_LED, LOW);
digitalWrite(RED_LED, HIGH);

dangerAlarm();

if (!alarmActive) {
alarmActive = true;
alarmStart = millis();
Serial.println("DANGER: PROLONGED EYE CLOSURE");
Serial.println("WAKE UP DRIVER!");
}
}
}

else {

if (eyeClosed) {
Serial.println("Eye opened");
}

eyeClosed = false;

noTone(BUZZER);

digitalWrite(RED_LED, LOW);
digitalWrite(YELLOW_LED, LOW);
digitalWrite(GREEN_LED, HIGH);

// Reset alarm state after the driver opens eyes
if (alarmActive &&
millis() - alarmStart > RESET_TIME) {
alarmActive = false;
Serial.println("System reset - Driver Alert");
}
}
```

```
// Print system status every 5 seconds
static unsigned long lastStatus = 0;

if (millis() - lastStatus >= 5000) {
lastStatus = millis();

Serial.print("System Status | Blinks/Closures: ");
Serial.print(blinkCount);

if (eyeClosed) {
Serial.println(" | EYE CLOSED");
} else {
Serial.println(" | EYE OPEN");
}
}

delay(50);
}
```